

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

Trabajo Práctico N° 3

TEMA: Programación en C. Estructuras de iteración.

En la vida cotidiana existen situaciones que frecuentemente se resuelven mediante una **secuencia de pasos que puede repetirse varias veces**. Por ejemplo:

- Seguir mezclando ingredientes mientras no se logre la consistencia deseada.
- Regar plantas hasta que el suelo esté húmedo.
- Revisar correos electrónicos hasta que se hayan leído todos los mensajes importantes.

El lenguaje C, tiene las siguientes estructuras:

- **while (MIENTRAS):** Evalúa la condición antes de ejecutar el bloque de instrucciones. Si la condición es falsa desde el inicio, el bloque no se ejecuta ninguna vez.
- **do/while (HACER-MIENTRAS):** Evalúa la condición después de ejecutar el bloque de instrucciones. Por eso, el bloque siempre se ejecuta al menos una vez. Se usa cuando se necesita ejecutar el ciclo al menos una vez, independientemente de la condición inicial.
- **for (HACER):** Se usa cuando se conoce el número de iteraciones que se desea realizar. Permite definir la variable de control, la condición de continuación y la actualización en la misma línea.



Consigna de trabajo

En todos los ejercicios deberá:

1. **Diseñar y escribir un algoritmo** en pseudocódigo que resuelva el problema planteado.
2. **Codificarlo en Lenguaje C**, utilizando de manera adecuada los tipos de datos disponibles y las estructuras de selección e iteración.
3. **Probar el programa** con distintos conjuntos de datos, para verificar su correcto funcionamiento y analizar posibles casos especiales.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

1. Analizando el número

Dado un número entero, determinar:

- a. La cantidad de dígitos que lo componen.
- b. Cantidad de dígitos primos. (Números primos entre 0 y 9: 2, 3, 5, 7).
- c. Suma de los dígitos primos.
- d. Suma de los impares.

Requisitos:

- El número debe estar comprendido entre 10 y 9999999.
- Para determinar si un dígito es primo no debe comparar directamente con los valores mencionados como dato en el inciso b.
- Como parte de las salidas muestre el número ingresado.

2. Palabras más, palabras menos

Diseñar un algoritmo que procese una secuencia de caracteres, la cual termina con una marca de fin de entrada.

A partir de esta secuencia se deben calcular y mostrar los siguientes datos:

- a. Cantidad de palabras.
- b. Longitud de la palabra más larga.
- c. Longitud de la palabra más corta.
- d. Promedio de caracteres por palabra.
- e. Porcentaje de espacios sobre el total de caracteres.

Requisitos:

- El algoritmo deberá ir leyendo y analizando cada carácter de manera individual.
- Mostrar el promedio y porcentaje con dos decimales.
- Realizar los ajustes/controles necesarios para que el algoritmo funcione correctamente según los casos de prueba.
- La secuencia solo debe incluir letras, números y espacios. Si un carácter no se encuentra dentro de los valores esperados se debe informar con un mensaje y detener la evaluación de la secuencia.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

Casos de prueba

Secuencia: *La inteligencia artificial es la columna vertebral de la innovación en la computación moderna.*

INFORME FINAL

- Cantidad de palabras: **14**
- Longitud de la palabra más larga: **12** (inteligencia/computación)
- Longitud de la palabra más corta: **2** (La/es/la/de/la/en/la)
- Promedio de caracteres por palabra: **5.71**
- Porcentaje de espacios sobre el total de caracteres: **13.98%**

Secuencia: *_ _ El _ _ _ lunes _ es _ día _ de _ _ práctica.*

INFORME FINAL

- Cantidad de palabras: **6**
- Longitud de la palabra más larga: **8** (práctica)
- Longitud de la palabra más corta: **2**
- Promedio de caracteres por palabra: **3.67**
- Porcentaje de espacios sobre el total de caracteres: **31.25%**

Secuencia: *_ _ _ _ .*

INFORME FINAL

- Cantidad de palabras: **0**
- Longitud de la palabra más larga: **0**
- Longitud de la palabra más corta: **0**
- Promedio de caracteres por palabra: **0**
- Porcentaje de espacios sobre el total de caracteres: **100%**

Secuencia: *.*

INFORME FINAL

- Cantidad de palabras: **0**
- Longitud de la palabra más larga: **0**
- Longitud de la palabra más corta: **0**
- Promedio de caracteres por palabra: **0**
- Porcentaje de espacios sobre el total de caracteres: **0%**

Secuencia: *El _ _ _ .*

INFORME FINAL

- Cantidad de palabras: **1**
- Longitud de la palabra más larga: **2**
- Longitud de la palabra más corta: **2**
- Promedio de caracteres por palabra: **2.00**
- Porcentaje de espacios sobre el total de caracteres: **60.00%**

Nota: El guión bajo representa un espacio en blanco en la secuencia.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

3. Cajero automático simplificado

Un cajero automático contiene inicialmente una cantidad determinada de billetes de **1000, 2000, 10000 y 20000 pesos**.

El programa debe inicialmente realizar la carga de cada tipo de billete y simular el proceso de retiros hasta que el cajero se quede sin dinero o hasta que se pare la simulación.

Para cada retiro:

- Verificar que el monto sea múltiplo de 1000. Si no lo es, mostrar un error y pedir otro monto.
- Intentar entregar el monto con los billetes disponibles (usando primero los de mayor valor).
- Si no alcanza el stock, informar que el retiro no puede realizarse.
- Si se puede, descontar los billetes usados del stock y mostrar el desglose de billetes entregados.

Al finalizar el proceso, informar si el cajero quedó sin dinero o se terminó la simulación por decisión del usuario.

Además, mostrar un **resumen final** con:

- Cantidad total de retiros realizados.
- Dinero total entregado.
- Stock final de cada tipo de billete en el cajero.